06 — MySQL (going to production)

Optional module: second day or self-study.

Module 05 used SQLite: perfect to learn, since it's one file and nothing to install. Production PHP sites almost always run on a **database server** instead, and in the PHP world that's mostly **MySQL** or its fork **MariaDB**. They're nearly interchangeable for everything in this course. WordPress, WooCommerce, Drupal, Magento and most shared hosting offers rely on them.

> **PostgreSQL** is the other big one, popular in Symfony and Laravel projects. With PDO, switching is mostly the connection string: `pgsql:host=127.0.0.1;dbname=training`.

The good news: **your PDO code doesn't change**. Same `prepare`, `execute`, `fetch`, transactions. What changes is everything around it: installing a server, users and passwords, column types, configuration.

SQLite vs MySQL

	SQLite	MySQL / MariaDB
What it is	a library writing to one file	a server program you connect to over the network
Users	none: file permissions	users, passwords, permissions per database
Concurrent writes	limited	built for many simultaneous users
Typical use	tests, prototypes, mobile apps, small sites	production web apps
PDO extension	`pdo_sqlite`	`pdo_mysql`

Installing a MySQL server

Option A — Docker (recommended: identical on every OS)

Install Docker Desktop (Windows, macOS) or Docker Engine (Linux). Then, in this folder:

```
docker compose up -d
```

This starts two things, defined in `docker-compose.yml`:

- a MySQL 8.4 server on port `3306`, with a database `training` and a user `training` / password `training`
- **Adminer**, a web interface to browse the database: http://localhost:8080 (System: *MySQL*, Server: `db`, user/password: `training`)

```
docker compose ps        # is it running?
docker compose logs db   # server logs (wait for "ready for connections")
docker compose down      # stop (data is kept in a volume)
```

> Port 3306 already used (an existing MySQL on your machine)? Change `"3306:3306"` to `"3307:3306"` in the compose file and use port 3307 in your DSN.

Option B — native install

OS	Install	Start
macOS	`brew install mysql`	`brew services start mysql`
Windows	MySQL Community Installer (choose "Server only")	runs as a Windows service
Linux	`sudo apt install mariadb-server`	starts automatically

Then create the database and user yourself (see *Users and permissions* below).

On Linux, MariaDB's `root` account logs in with `sudo mysql` (no password, uses your system account).

Enable `pdo_mysql` in PHP

```
php -m | grep -i pdo_mysql        # Windows: php -m | findstr pdo_mysql
```

If nothing shows: Windows → uncomment `extension=pdo_mysql` in `php.ini`; Linux → `sudo apt install php-mysql`. Homebrew's PHP already includes it.

Talking to the server

The command-line client:

```
docker compose exec db mysql -u training -p training    # Docker (password: training)
mysql -u training -p training                            # native install
```

```
SELECT VERSION();
SHOW DATABASES;
USE training;
SHOW TABLES;
DESCRIBE books;
exit
```

Or use a GUI: Adminer (included in the Docker setup), phpMyAdmin, DBeaver, TablePlus, or the VS Code extension *Database Client*.

To **design** tables as a diagram and generate the SQL, use MySQL Workbench (free). More in useful tools.

Users and permissions

Never let your application connect as `root`. Create one user per application, with rights on its own database only:

```
CREATE DATABASE training CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'training'@'%' IDENTIFIED BY 'training';
GRANT ALL PRIVILEGES ON training.* TO 'training'@'%';
```

(The Docker setup already did this for you.)

Writing tables for MySQL

MySQL is stricter about types than SQLite. A typical table:

```
CREATE TABLE books (
    id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title      VARCHAR(255) NOT NULL,
    price      DECIMAL(8,2) NOT NULL DEFAULT 0,      -- money: DECIMAL, never FLOAT
    published  DATE NULL,
    in_stock   BOOLEAN NOT NULL DEFAULT TRUE,        -- stored as TINYINT(1)
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

SQLite	MySQL
`INTEGER PRIMARY KEY AUTOINCREMENT`	`INT UNSIGNED AUTO_INCREMENT PRIMARY KEY`
`TEXT` for any string	`VARCHAR(n)` for short strings, `TEXT` for long ones
`REAL`	`DECIMAL(p,s)` for exact values (money), `DOUBLE` otherwise
dates stored as text	`DATE`, `DATETIME`, `TIMESTAMP`

- **`utf8mb4`**, not `utf8`: MySQL's old `utf8` can't store emojis and some characters. `utf8mb4` is the real UTF-8.
- **InnoDB** is the engine that supports transactions and foreign keys (default in modern versions).

Foreign keys and joins

A real database has several related tables. A foreign key guarantees that a book always points to an existing author:

```sql
CREATE TABLE authors (
    id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE books (
    id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title     VARCHAR(255) NOT NULL,
    author_id INT UNSIGNED NOT NULL,
    FOREIGN KEY (author_id) REFERENCES authors(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```sql
-- Each book with its author's name
SELECT b.title, a.name AS author
FROM books b
JOIN authors a ON a.id = b.author_id
ORDER BY a.name, b.title;

-- Number of books per author
SELECT a.name, COUNT(b.id) AS nb_books
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
GROUP BY a.id, a.name;
```

Keep your schema in a `.sql` file in the project, and load it with:

```bash
docker compose exec -T db mysql -u training -ptraining training < schema.sql   # Docker
mysql -u training -p training < schema.sql                                      # native
```

Connecting from PHP

```php
$pdo = new PDO(
    'mysql:host=127.0.0.1;port=3306;dbname=training;charset=utf8mb4',
    'training',
    'training',
    [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,   // real server-side prepared statements
    ]
);
```

Three details that save hours:

- **`127.0.0.1`, not `localhost`**: on macOS/Linux, `localhost` makes PHP look for a local socket file instead of using the network, which fails with Docker ("*No such file or directory*").
- **`charset=utf8mb4`** in the DSN, otherwise accents can be mangled.
- **`ATTR_EMULATE_PREPARES => false`**: by default PDO-MySQL *simulates* prepared statements and sends every value as a string. One visible symptom: `LIMIT ?` with `execute([5])` becomes `LIMIT '5'` and fails. Turn emulation off.

Keep credentials out of the code

Passwords don't belong in files you commit to Git. A simple, common approach:

```php
// config.php — NOT committed (listed in .gitignore)
return [
    'dsn'      => 'mysql:host=127.0.0.1;dbname=training;charset=utf8mb4',
    'user'     => 'training',
    'password' => 'training',
];
```

```php
// config.example.php — committed, same keys, fake values, so others know what to fill in
```

```php
$config = require __DIR__ . '/config.php';
$pdo = new PDO($config['dsn'], $config['user'], $config['password'], $options);
```

Frameworks do the same with a `.env` file (`DATABASE_URL=mysql://...`).

Common connection errors

Message contains	Meaning	Fix
`could not find driver`	`pdo_mysql` not enabled	see *Enable pdo_mysql*
`[2002] No such file or directory`	`localhost` → socket	use `127.0.0.1`
`[2002] Connection refused`	server not running / wrong port	`docker compose ps`, check port
`[1045] Access denied`	wrong user or password	check credentials
`[1049] Unknown database`	database doesn't exist	`CREATE DATABASE` / check name

Backups

```
docker compose exec -T db mysqldump -u training -ptraining training > backup.sql   # export
docker compose exec -T db mysql -u training -ptraining training < backup.sql    # restore
```

(Native install: same commands without `docker compose exec db`.)

Now do the exercises.

06 — Exercises

6.1 — Start and explore the server

1. Start MySQL (Docker or native, see the lesson).
2. Connect with the command-line client and run `SELECT VERSION();` and `SHOW DATABASES;`.
3. Open Adminer (http://localhost:8080) or another GUI and log into the `training` database.
4. Check that PHP has the driver: `php -m` must list `pdo_mysql`.
5. Write `connect.php` that connects with PDO and displays `Connected to MySQL x.y.z` using `SELECT VERSION()`.
6. Break it on purpose, one at a time, and read the error: wrong password, wrong database name, `localhost` instead of `127.0.0.1` (macOS/Linux with Docker), server stopped.

6.2 — Library with two tables

1. Write `schema.sql` creating two tables:

   - `authors` (id, name)
   - `books` (id, title, year, price as `DECIMAL(6,2)`, author_id with a foreign key to `authors`)

   Start the file with `DROP TABLE IF EXISTS books, authors;` so you can reload it as often as you want. Insert at least 3 authors and 6 books.

2. Load it into MySQL (command line or Adminer's *Import*).

3. Write `library.php` (command line) that:

   - lists every book with its author's name, sorted by author then year (`JOIN`)
   - displays the number of books and the average price per author (`GROUP BY`, `COUNT`, `AVG`)
   - asks for an author name as command-line argument (`php library.php Orwell`) and lists only their books, with a prepared statement and `LIKE`. Hint: `$argv[1]`.

4. Try inserting a book with an `author_id` that doesn't exist. What happens, and why is it a good thing?

6.3 — Move the guestbook to MySQL

Take your guestbook from exercise 5.3 (or the provided solution) and make it run on MySQL.

1. Write `schema.sql` for the `messages` table with proper MySQL types (`VARCHAR`, `TEXT`, `DATETIME DEFAULT CURRENT_TIMESTAMP`).
2. Create `config.php` (credentials) and `config.example.php` (same keys, fake values). Add `config.php` to a `.gitignore`.
3. Change `Database.php` to read the config and connect to MySQL. Remove the `CREATE TABLE` from the PHP code: the schema now lives in `schema.sql`.
4. Run the app. **Nothing else should need to change**: `MessageRepository`, `index.php` and `admin.php` stay as they are. That's the point of PDO and of keeping all SQL in one class.

Bonus 1: the guestbook paginates with `bindValue(..., PDO::PARAM_INT)` in `MessageRepository::findPage()`. Replace it with a plain `$stmt->execute([$limit, $offset])` and try with `ATTR_EMULATE_PREPARES` set to `true`, then `false`. What happens, and why? (Answer in the *Connecting from PHP* section of the lesson.)

Bonus 2: move the bookshop app (final project) to MySQL the same way: `config.php`, a `schema.sql` with proper MySQL types (`DECIMAL(6,2)` for the price), and a new `Database.php`. The repository, the validator and all the pages should stay unchanged.